Code559

Front-End Developer Course

MODULE 5

JavaScript Primer

Run your first script, read your first error, and build the habit you will use for every line of code.

Free lesson: 559solutions.com/code559/module-05-javascript-primer/

In this packet

1. **The lesson**
2. **Project: About me in the Console**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn what a program is, where JavaScript runs, how to see a value and how to read an error message. You also start the Predict → Run → Modify → Create → Debug habit. It's the habit every later module depends on: run it, change it, break it, and read what the computer tells you.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 5
- Core concepts 5
  - 1. What is a program? 5
  - 2. Where does JavaScript run? 6
  - 3. What are statements, semicolons and comments? 6
  - 4. How do you see a value? 7
  - 5. How do you run a file with Node? 7
  - 6. How do you open the Console and try an expression? 8
  - 7. How do you read an error message? 9
  - 8. What is the habit? 10
- Exercise 11
- Common misconception 13
- Summary 13
- Check your understanding 14
- Before next module 14
- Go further 14
- What to keep 14

Part 2: Project: About me in the Console 16

Part 3: Finishing work in Git 23

Part 4: Troubleshooting: five common problems 25

Part 5: Self-assessment 28

Part 6: Answers 30

About 559 Solutions 33

© 2026 559 Solutions LLC

PART 1

The lesson

Run your first script, read your first error, and build the habit you will use for every line of code.

Time: about 75 minutes (estimate)

In this module you will learn:

- What a program is: instructions that run in order, top to bottom
- The two places JavaScript runs in this course: the browser and Node
- Where JavaScript fits next to HTML and CSS (structure, appearance, behavior)
- What a statement is, what the semicolon does, and how to write a comment
- How `console.log()` lets you see a value
- How to run a `.js` file with `node file.js` in the terminal
- How to open the browser's DevTools Console and type an expression
- How to read an error message: its name, its message, and its file and line
- The habit for every piece of code: Predict → Run → Modify → Create → Debug

---

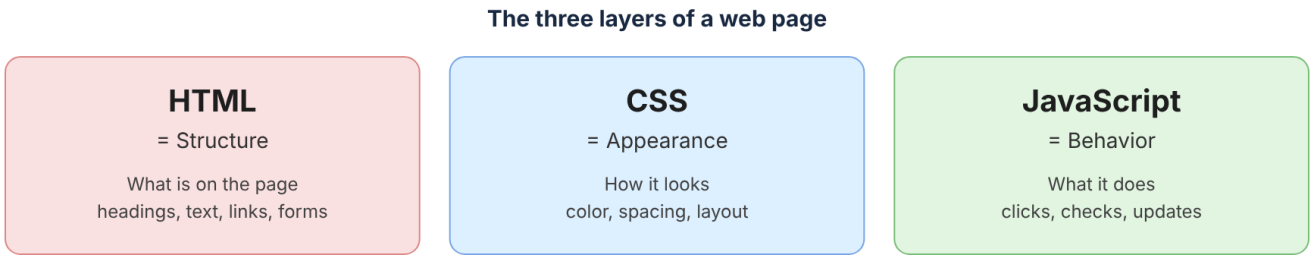
Why this matters

Up to now your pages have stood still. HTML put things on the page and CSS made them look right. JavaScript is the part that makes a page *do* something. Before it can do anything on your site, you need to be comfortable with the language on its own, with no page in the way.

That is what Modules 5 to 8 are for. You will write JavaScript and see the results as plain text, in the terminal and in the browser's Console. Nothing on your site changes yet. You will connect JavaScript to your pages in Module 9.

This module is short on new syntax and long on habits. You will run a script, change it, break it on purpose, and read what the computer tells you. Every later module depends on that loop.

© 2026 559 Solutions LLC



HTML is what is on the page, CSS is how it looks, JavaScript is what it does. MDN's "What is JavaScript?" page calls these "the three layers of standard web technologies."

Before you start

1. Finish Module 0. Node must be installed: in a terminal, `node --version` should print a version starting with `v`.
2. In your course folder (for example `Documents\code559`), create a new folder called `practice-05`. This is a scratch folder for practice files. It sits next to `my-site`, not inside it.
3. Open `practice-05` in VS Code with **File → Open Folder...**, then open a terminal with **Terminal → New Terminal**.

> **Checkpoint.** Your terminal prompt ends in `practice-05>` and `node --version` prints a version.

Core concepts

1. What is a program?

A program is a list of instructions for the computer. In JavaScript, each instruction is usually one line.

The computer runs the instructions **in order, from top to bottom**, one at a time. It does not skip ahead or guess what you meant. If line 2 has a mistake, the computer stops at line 2. Later modules add ways to repeat lines or choose between them, but the starting rule is always the same: top to bottom.

In the next concept you will run a three-line script. The three lines of output come out in the same order as the three lines of code.

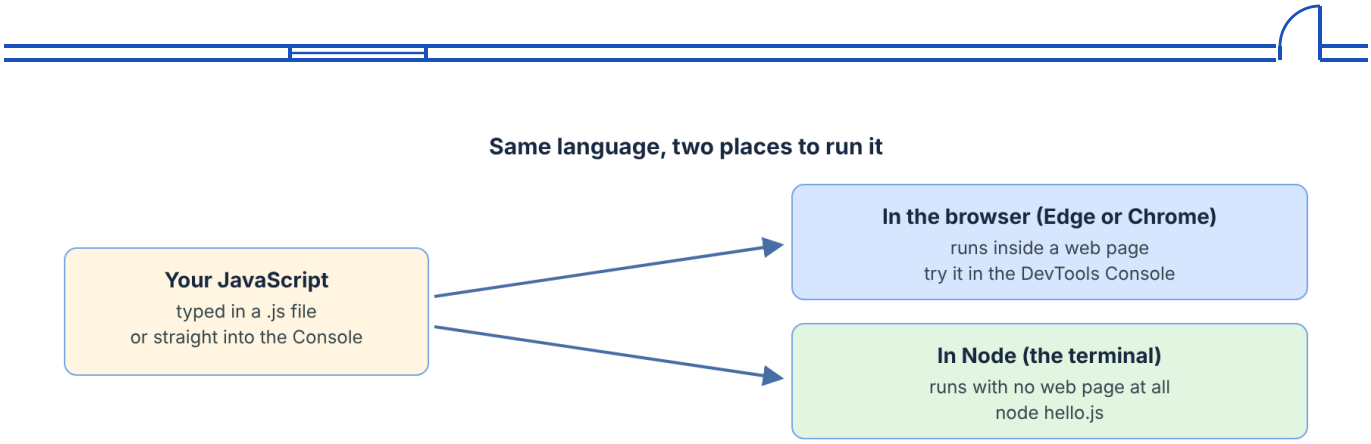
Write three things you did this morning as three numbered steps. Swap two of them. Did the meaning change? Order matters to a program in the same way.

© 2026 559 Solutions LLC

2. Where does JavaScript run?

JavaScript was made for web browsers. Every browser, including Edge and Chrome, can run it. **Node** is a program that runs JavaScript outside a browser, in the terminal.

The language is the same in both places. The browser runs JavaScript as part of a web page; you will use its **Console** to try one line at a time. Node runs a whole `.js` file from the terminal, with no web page at all. In this module you use both: Node for files, the Console for quick experiments.



Same language, two places to run it. Modules 5 to 8 use both; Module 9 brings JavaScript into your pages.

Write down which place you would use to (a) try `2 + 3` quickly, (b) run a saved file called `hello.js`. (Answers: the Console; Node.)

3. What are statements, semicolons and comments?

A **statement** is one complete instruction, like a sentence. A **comment** is a note for humans that the computer skips.

In this course every statement ends with a semicolon `;`, the way a sentence ends with a period. JavaScript can often add a missing semicolon for you (MDN's *Lexical grammar* page calls this "automatic semicolon insertion"), but writing them yourself keeps your code clear. A comment starts with `//`; MDN puts it this way: it "makes all text following it on the same line into a comment." Use comments to explain *why* a line is there.

Here is the first script. Line 1 is a comment. Lines 2 to 4 are statements.

```
// hello.js: my first script
console.log("Hello, world!");
console.log("This line runs second.");
console.log(2 + 3); // a number, not text
```


© 2026 559 Solutions LLC

Point at each semicolon in lines 2 to 4 and explain it to yourself, out loud: "This semicolon ends the instruction on this line."[1] Say it three times, once per line. Your memory aid is the short form, **"semicolon: end of instruction"**; recite it until it comes without thinking. Then find the comment at the end of line 4 and explain it out loud too: "Everything after the two slashes is a note the computer skips."

4. How do you see a value?

`console.log()` prints whatever you put between the parentheses.

Text goes inside quotes: `"Hello, world!"`. A sum like `2 + 3` has no quotes, so JavaScript works it out first and prints the result, `5`. In Node, `console.log()` prints to the terminal. In the browser, it prints to the Console. It is the main way you will check your work in Modules 5 to 8: when you are unsure what a value is, log it.

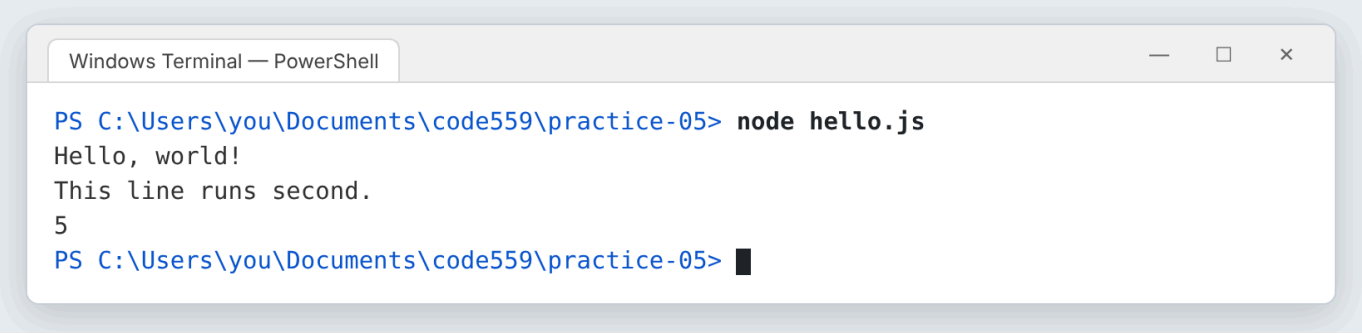
You will see the output in the next concept.

Predict what `console.log(10 - 4);` prints. Then predict what `console.log("10 - 4");` prints. (One works out the sum; the other prints its characters exactly as typed. You will check both in the Exercise.)

5. How do you run a file with Node?

In the terminal, `node` followed by a file name runs that file.

Type `node hello.js` and press Enter. Node reads the file from top to bottom and runs each statement. Anything you `console.log()` appears in the terminal, and then the prompt comes back. The terminal must be in the folder that holds the file. If it is not, Node prints `Error: Cannot find module` and the path it tried.



Three statements, three lines of output, in the same order. The comment printed nothing. On Windows the output is the same. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

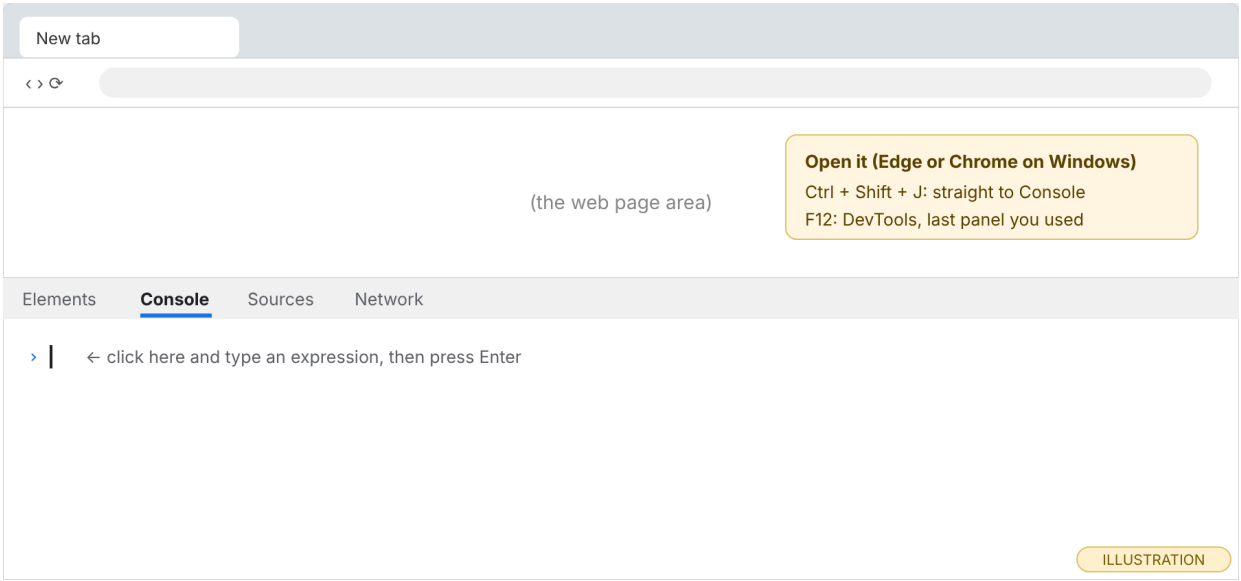
© 2026 559 Solutions LLC

Look at the third output line. Why does it show `5` and not `2 + 3`?

6. How do you open the Console and try an expression?

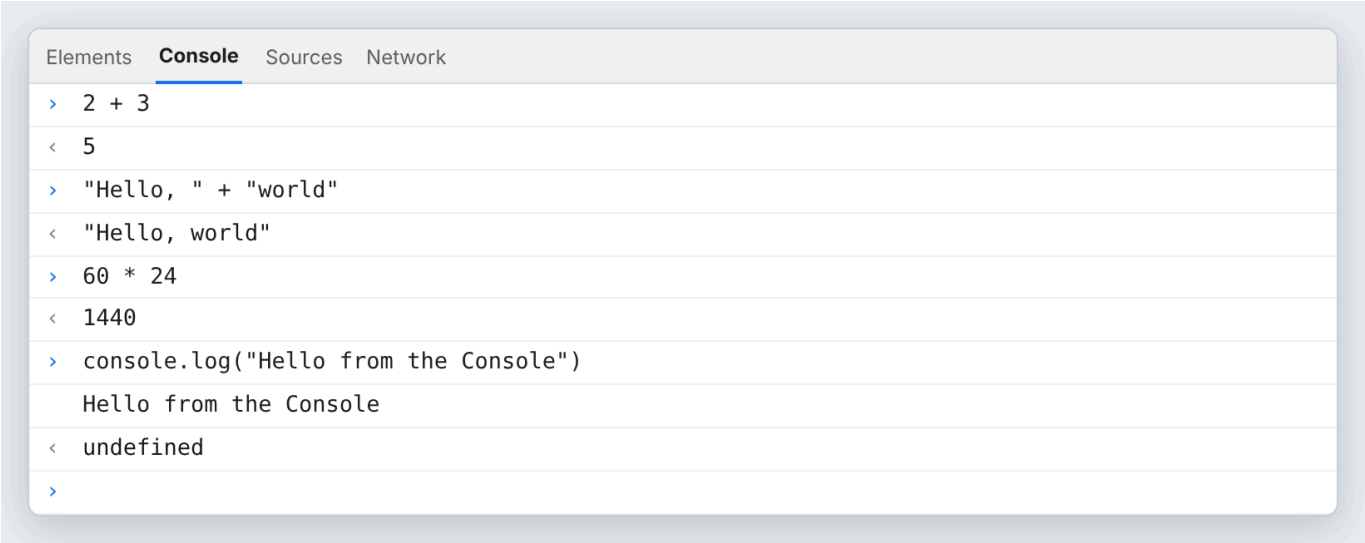
Every Edge and Chrome window has built-in developer tools, called **DevTools**. The **Console** is one of its panels.

Open `index.html` from your `my-site` folder (or any other page) in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. (Both shortcuts are listed in Chrome's *Open Chrome DevTools* page and Microsoft's *Keyboard shortcuts* page for Edge DevTools.) Click next to the `>` sign at the bottom, type an expression, and press Enter. The Console shows the result on the next line.



DevTools usually opens docked to the side or bottom of the window. Type next to the `>` sign.

© 2026 559 Solutions LLC



Each expression gets an answer line. `console.log()` prints its text, then shows `undefined`: that is the Console reporting that `console.log()` itself hands back no value. It is not an error. Your Console may show text in single quotes instead of double; both mean text. (real output, rendered; evaluated in Chromium)

Predict first. Write your prediction down before you type. In Edge and Chrome, the Console shows a gray preview of the result under the line you are typing, before you press Enter. This feature is called **Eager evaluation**, and it is on by default. If you want to predict first, turn it off: click the **Console settings** gear in the top-right corner of the Console and clear the **Eager evaluation** checkbox. (Chrome's and Microsoft's *Console features reference* pages both describe this setting.) One more thing: if you paste code into the Console, Edge or Chrome may block it and ask you to type `allow pasting` first. This warning protects people from pasting harmful code they do not understand. Typing the code yourself avoids it.

Open the Console now and type `60 * 24`. The `*` sign means multiply, so that is the number of minutes in a day.

7. How do you read an error message?

When JavaScript cannot run a line, it stops and prints an **error**. An error is not a punishment. It is a report.

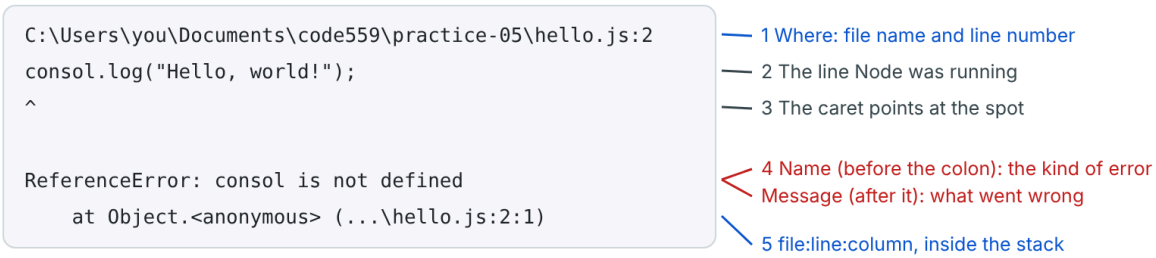
MDN's *JavaScript error reference* says each error "has a `name` and a `message`." A Node error has five parts. Read them in the same order every time, from the top:

1. **Where:** the file name and line number at the top.
2. **The line:** the code Node was running.
3. **The caret:** the `^` under the problem spot.
4. **Name and message:** the name is the word before the colon, such as `ReferenceError` (a name JavaScript does not recognize) or `SyntaxError` (the code is not valid JavaScript, so nothing in the file ran). The message is the words after the colon.

© 2026 559 Solutions LLC

5. **The stack:** the lines starting with `at`. The first one repeats your file as `file:line:column`, and the rest point inside Node itself, so you can usually skip them. In a `SyntaxError`, every `at` line points inside Node, because your file never started running.

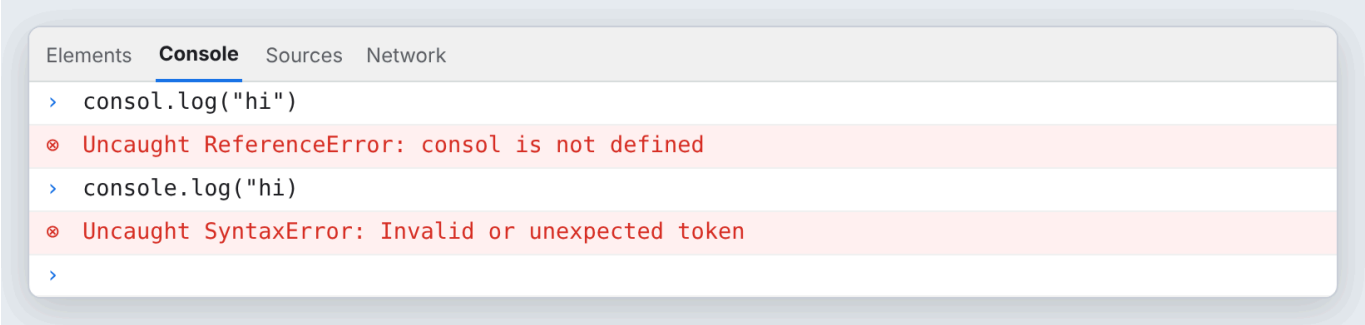
Reading a Node error from the top



Read the top of the message first. The other "at ..." lines are Node's own files; you can usually skip them.

Read from the top down. This is the real error from the Exercise's break-it step, with the folder path changed to the course folder and shortened to `...\` on the stack line.

In the Console, the same mistake shows a shorter error. It starts with `Uncaught`, which only means no code caught the error before it reached you.



Same names, same messages as in Node. The real Console also adds a small location link next to each error, and under the ReferenceError one stack line, `at <anonymous>:1:1` (line 1, column 1 of what you typed); this render shows only the name and message. (real output, rendered; evaluated in Chromium)

In the Console, type `consol.log("hi")` with the missing `e`. Find the error's name and its message, and write them down.

8. What is the habit?

Every piece of code in this course goes through five steps: **Predict, Run, Modify, Create, Debug.**

© 2026 559 Solutions LLC

Predict: before you press Enter, write down what will happen. **Run:** run it and compare with your prediction. **Modify:** change one thing and predict again. **Create:** write a line of your own that does something similar. **Debug:** break it on purpose and read the message. The prediction is the important part. When your prediction is wrong, you have found exactly the thing you did not understand yet.



The habit for every piece of code from here on.

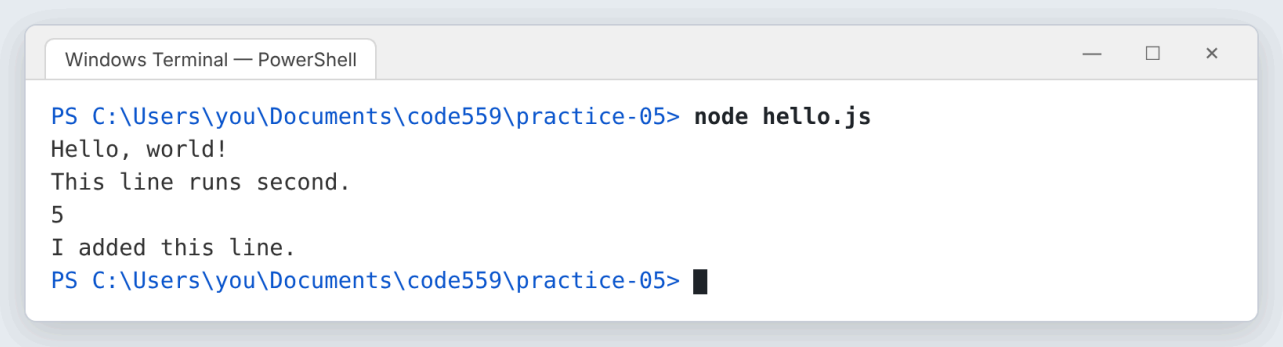
You will use all five steps in the Exercise, in order.

Exercise

Your hands, about 30 minutes (estimate). Type it, do not paste it. Work in `practice-05`, your scratch folder.

1. **Create the file.** In VS Code, create `hello.js` in `practice-05`. Type the four lines from Core Concept 3 exactly. Save with Ctrl+S.
2. **Predict.** Before you run it, write down the three lines you expect to see, in order.
3. **Run.** In the terminal, type `node hello.js` and press Enter. Compare with your prediction.
4. **Modify.** Add a fifth line at the bottom: `console.log("I added this line.");` Save. Predict, then run again.

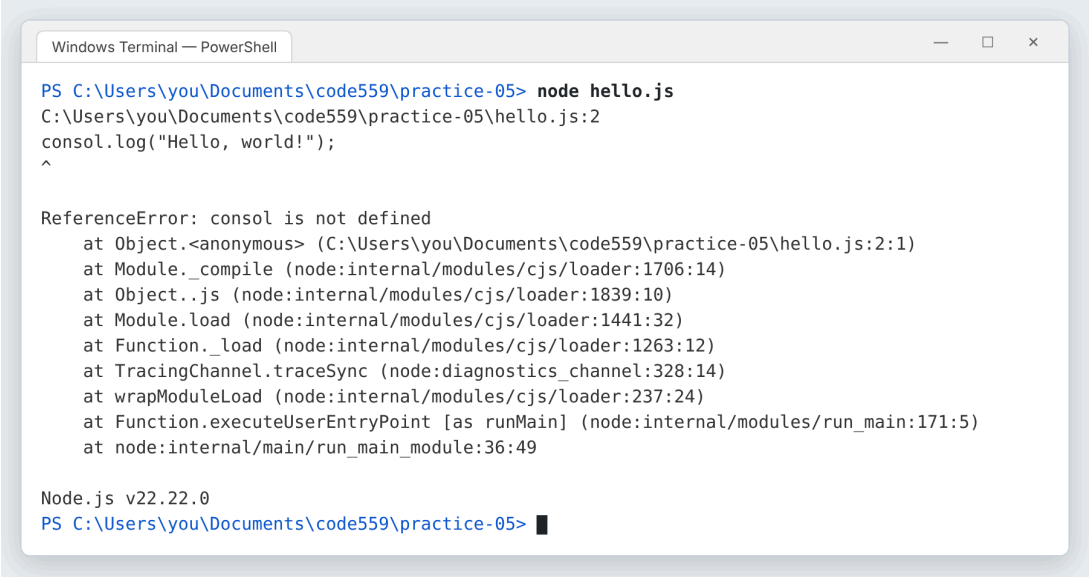
© 2026 559 Solutions LLC



The new line appears last, because it is the last statement in the file. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

5. **Create.** Add a line of your own that logs a multiplication with `*`, for example the number of hours in a week. Predict the number first. Then add `console.log(10 - 4);` and `console.log("10 - 4");` and check your prediction from Core Concept 4.

6. **Break it on purpose: a misspelled name.** On line 2, delete the `e` so it reads `consol.log("Hello, world!");`. Save and run.

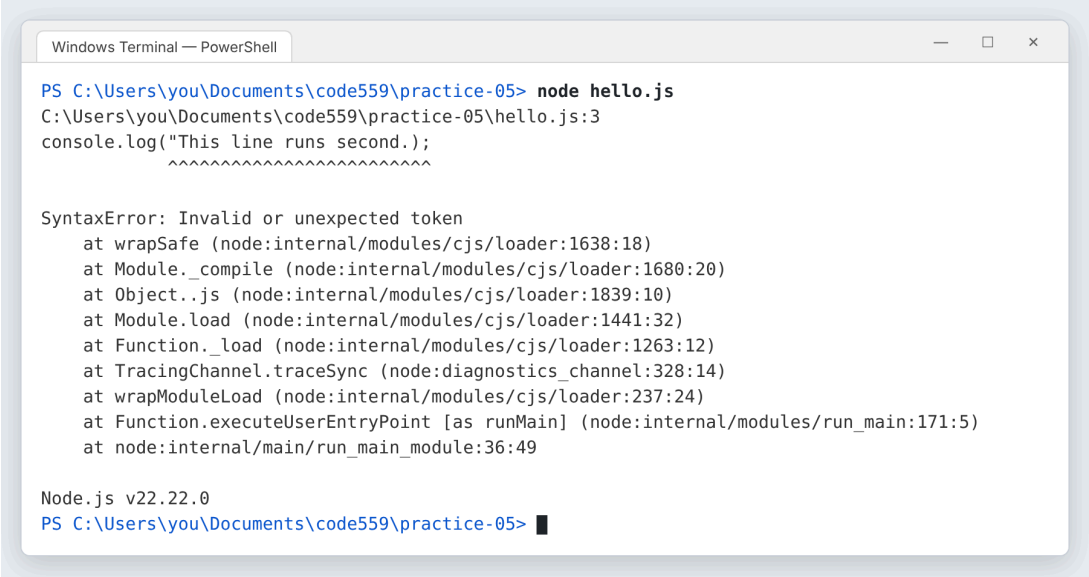


Node stopped at line 2, so nothing printed, not even the lines after it. Name: `ReferenceError`. Message: `consol is not defined`. The first line and the first `at` line show the path to your own `hello.js`. On Windows the paths start with your drive, for example `C:\Users\…`. The `node:internal` lines and the last line, `Node.js v22.22.0`, depend on the Node version, so yours may show different lines and numbers. With Node 24.21.0, everything down to the first `at` line was the same. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Find the five parts from Core Concept 7. Fix the spelling, save, and run again to prove it works.

© 2026 559 Solutions LLC

7. **Break it again: a missing quote.** On line 3, delete the closing quote so it reads `console.log("This line runs second.);`. Save and run.



A `SyntaxError` is different: Node could not read the file at all, so not even line 2 printed. The carets underline everything from the open quote to the end of the line, because Node is still looking for the closing quote. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Fix it, save, and run once more.

8. **Try the Console.** Open any page in Edge or Chrome, press **F12** (or **Ctrl+Shift+J**), click the **Console** tab, and type `2 + 3`, then `"Hello, " + "world"`, then `consol.log("hi")`. Compare with the Console images in Core Concepts 6 and 7.

Common misconception

> *"An error message means I did something badly wrong. I should delete the line and start over."*

What would you say? Hint: count how many useful facts the ReferenceError in step 6 gave you, for free, before you had to look anything up.

Summary

A program is a list of instructions that run in order, top to bottom. JavaScript runs in the browser and, through Node, in the terminal. HTML is structure, CSS is appearance and JavaScript is behavior. A statement is one instruction ending in a semicolon, and `//` starts a comment the computer skips. `console.log()` prints a value so you can see it. `node file.js` runs a file from the terminal in the file's

© 2026 559 Solutions LLC

folder. Ctrl+Shift+J opens the DevTools Console, where you type an expression and press Enter. A Node error has five parts, read from the top: where (file and line), the code line, the caret, the name and message (what kind of error and what went wrong), and the `file:line:column` in the stack. Predict → Run → Modify → Create → Debug is the habit for every piece of code.

Check your understanding

1. In step 6, why did nothing print at all, even though the lines after line 2 were fine?
2. What is the difference between `console.log(10 - 4);` and `console.log("10 - 4");`?
3. **Why does it work?** You typed `console.log("Hello from the Console")` in the Console and saw the text and then `undefined`. Why does the `undefined` appear, and why is it not an error?

Before next module

- [ ] `hello.js` runs with `node hello.js` and prints the lines you predicted
- [ ] You can open the Console with Ctrl+Shift+J and evaluate an expression
- [ ] You broke `hello.js` twice and can name the error each time (ReferenceError, SyntaxError)
- [ ] You can point to the five parts of a Node error
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *What is JavaScript?* (Learn web development)
- MDN: *JavaScript error reference*, and its pages for *ReferenceError* and *SyntaxError*
- MDN: *Lexical grammar* (the sections on comments and automatic semicolon insertion)
- Chrome for Developers: *Open Chrome DevTools*
- Microsoft Edge Developer documentation: *Keyboard shortcuts*
- Chrome for Developers and Microsoft Edge Developer documentation: *Console features reference* (the Eager evaluation setting)
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep `hello.js` in `practice-05`, and write down the names of the two errors you caused and what fixed each one. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

© 2026 559 Solutions LLC

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "What is JavaScript?"; MDN "JavaScript error reference"; MDN "Lexical grammar"; Chrome for Developers "Open Chrome DevTools"; Microsoft Edge Developer documentation "Keyboard shortcuts". Checked 2026-10-11: Chrome for Developers "Console features reference" and "How Chrome DevTools helps to defend against self-XSS attacks"; Microsoft Edge Developer documentation "Console features reference" and "Run JavaScript in the Console". Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: About me in the Console

Your hands, about 60 minutes (estimate). In the Exercise you ran, changed and broke `hello.js` in `practice-05`. Now you write a script of your own and keep it in your real site. The script is `about-me.js`. When you run it with Node, it prints a short introduction in labeled lines, like a small card in the terminal: who you are, what you are learning, and one thing you want to build. When Node runs the file, `console.log()` prints in the terminal, which is Node's console. You plan it with comments first and predict the output before every run. You use only what Module 5 taught: comments, statements, semicolons, text in quotes and `console.log()`.

This project goes in `my-site`, the folder you have built since Module 1. Any `my-site` that is a Git repository connected to GitHub works, because the script does not touch your pages. If you did the Module 4 packet, your site has five pages; this project adds a file that is not a page.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds a new file, `about-me.js`, next to `index.html` and your other pages. Not inside `practice-05`, and not in a sub-folder. No page links to it and no page runs it. In Modules 5 to 8 your JavaScript runs in the terminal only. Module 9 shows how a page loads a script.

What it prints. Run from the `my-site` folder, `node about-me.js` printed exactly these 18 lines for the sample student, Sam:

```
==============================
ABOUT ME
==============================
Name: Sam
Likes: drawing, soccer and old maps

------------------------------
WHAT I AM LEARNING
------------------------------
Course: Code559 Front-End Developer Course
So far: HTML, CSS and JavaScript
This week: running scripts with Node

------------------------------
ONE THING I WANT TO BUILD
------------------------------
Goal: a page where my family shares recipes
==============================
```


© 2026 559 Solutions LLC

Your script prints the same shape with your own words:

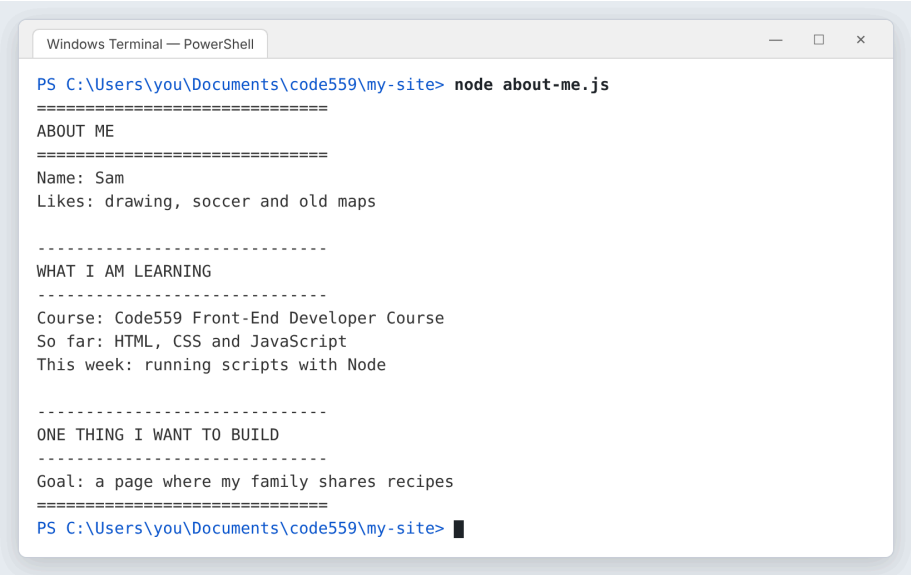
- **Three sections, in this order:** `ABOUT ME`, `WHAT I AM LEARNING`, `ONE THING I WANT TO BUILD`. Each heading is in capital letters, with a divider line above it and below it.
- **The dividers.** The top of the card and the line under `ABOUT ME` are `=` signs. The dividers in the two later sections are `-` signs. The very last line is `=` signs again: the bottom of the card. Every divider has the same length. The sample uses 30 characters.
- **Labeled lines.** Every fact is one line: a short label, a colon, a space, then your words, such as `Name: Sam`. `ABOUT ME` has your name and at least one more fact. `WHAT I AM LEARNING` has at least two lines. `ONE THING I WANT TO BUILD` has one `Goal:` line.
- **One blank line between sections**, before each section's first divider.

What is inside the file.

- The first two lines are comments: the file name with one sentence about what it does, and how to run it.
- Each section starts with a comment that names it, such as `// Section 1: who I am`.
- Every printed line, the blank ones too, comes from its own `console.log()` statement, and every statement ends with a semicolon.
- Only text in quotes. You learn to store values in Module 6.

The script will be public: GitHub Pages publishes every file in `my-site`, so anyone with the address can read it (Part 3 shows you). Write only what you are happy to share. Your first name or a nickname is enough. No address, no phone number, no school.

© 2026 559 Solutions LLC



The finished script for the sample student. Eighteen statements, eighteen lines of output, in the same order as the code. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

The steps

Type every line. Do not paste. The one exception is the divider lines: once you have typed one divider and checked its length, you may copy that whole line for the others.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`. Type `node --version`. It should print a version starting with `v`, as in Module 0.

2. **Plan on paper.** Read the brief again. On paper, write the 18 or so lines you want your script to print, in order, with your own words. Mark where each blank line goes. This is your plan, and later it is your prediction: when you run the finished script, the terminal should match this page line for line.

3. **Plan with comments first.** In the VS Code Explorer, create `about-me.js` at the top level of `my-site`, next to `index.html`. Type only these comments, then save with Ctrl+S:

```
// about-me.js: a short introduction that Node prints in the terminal
// Run it from the my-site folder with: node about-me.js

// Section 1: who I am

// Section 2: what I am learning

// Section 3: one thing I want to build
```


© 2026 559 Solutions LLC

The comments are the outline of the script. Each section's code goes under its own comment. **Predict** before you run it: what will `node about-me.js` print? Write down your answer. Then type `node about-me.js` and press Enter.

Nothing printed. The prompt came straight back, with no error. Node read every line, and every line was a comment, so there was nothing to do.

4. **Section 1.** Under `// Section 1: who I am`, type the five statements below, with your own name and your own fact in place of Sam's:

```
// Section 1: who I am
console.log("============================");
console.log("ABOUT ME");
console.log("============================");
console.log("Name: Sam");
console.log("Likes: drawing, soccer and old maps");
```

   Read the five statements to yourself, out loud. For each one, say what it will print. Then write the five lines down as your prediction. Save, run `node about-me.js` again, and compare.

   Save before every run. Node runs the file as it is saved on the disk, not what you see in the editor. A white dot on the file's tab in VS Code means the file has changes that are not saved yet.

5. **Section 2, with a blank line.** Under `// Section 2: what I am learning`, start with these two statements:

```
// Section 2: what I am learning
console.log("");
console.log("----------------------------");
// Continue alone: the heading WHAT I AM LEARNING, a second divider,
// then at least two labeled lines of your own.
```

   `""` is text with nothing in it: two quotes and no characters between them. `console.log("")` prints that empty text, so the terminal shows an empty line. That is your blank line between sections.

   Write the rest of the section alone. Then delete the two "Continue alone" comment lines, because they described work that is now done. Before you run, count your statements in Sections 1 and 2, and write down how many lines will print. Save and run. The sample printed 12 lines: 5 from Section 1 and 7 from Section 2, the empty one included.

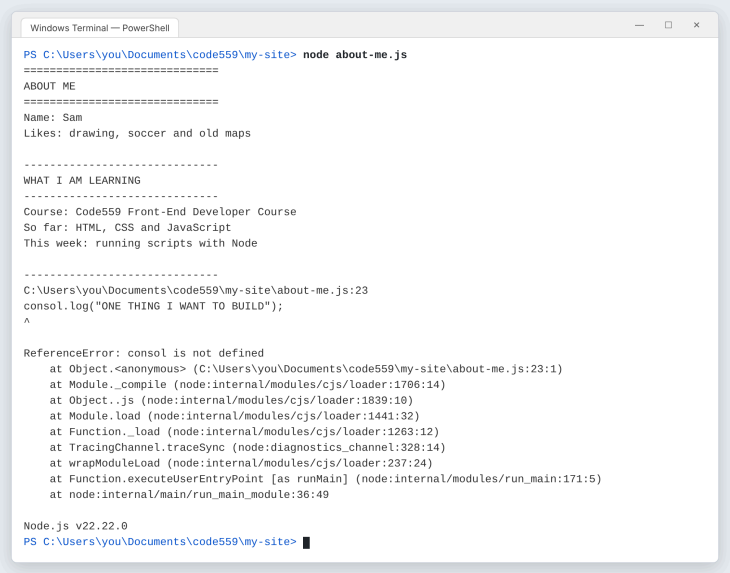
6. **Create Section 3 alone.** This section has no code to copy, only this description from the brief: a blank line, a `-` divider, the heading `ONE THING I WANT TO BUILD`, a second `-` divider, one `Goal:` line, then the bottom of the card, a line of `=` signs. Type it under `// Section 3: one thing I want to build`. Count all your statements and write down the number of lines you

© 2026 559 Solutions LLC

expect. Save and run. Compare the result with your plan from step 2 and with the picture in the brief. If a line is missing, in the wrong place or spelled differently, fix the statement that prints it and run again.

7. **Modify.** Make two changes, one at a time. Predict each before you run it.
    - **Remove the first blank line.** Put the cursor on the `console.log("");` line in Section 2 and press Ctrl+X to cut the whole line. Save. Predict what changes, then run. In the sample, the dashed divider now sat right under `Likes: drawing, soccer and old maps`, with no space between the sections. Put the cursor back at the start of the line under `// Section 2: what I am learning`, press Ctrl+V, save, and run again to check that the blank line is back.
    - **Swap two sections.** Select all of Section 3, from its comment to the last line of the file, and cut it with Ctrl+X. Paste it on the empty line above `// Section 2: what I am learning`. Save. Before you run, write down the order of the 18 lines. Then run. In the sample, the building section came second, and so did the closing line of `=` signs, because that statement was cut together with Section 3. The bottom of the card now sat in the middle of the output, and the card had no bottom at all. The computer did exactly what the file said, top to bottom. Move Section 3 back to the end, save, and run once more to check that the output matches the brief again.
8. **Break it on purpose: a misspelled name.** In Section 3, on the heading line, delete the `e` from `console`, so it reads `consol.log("ONE THING I WANT TO BUILD");`. Save. Before you run, think this through: in the Exercise, a typo on line 2 stopped everything, so nothing printed. This typo is near the end of the file. What will print this time? Write it down, then run.

© 2026 559 Solutions LLC



The first 14 lines printed: Node ran lines 5 to 22 before it reached the typo on line 23. Then it stopped, and the heading, the Goal line and the bottom of the card never printed. Your line number may differ if your sections have more or fewer lines. The `node:internal` lines and the last line depend on your Node version. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Find the five parts from the lesson's Core Concept 7: where (the file and line), the line, the caret, the name and message, and the `file:line:column` in the first `at` line. Fix the spelling, save, and run again to prove that all 18 lines are back.

9. **Debug: a message to read before you look.** Now a different mistake. On the `Goal:` line, delete the closing parenthesis `)`, the one just before the semicolon. Save. Before you run, write down one prediction: will the first 17 lines print this time, or none of them? Then run. Here are the top lines of what Node printed for the sample, with the folder path changed to the course folder (captured on Linux):

```
C:\Users\you\Documents\code559\my-site\about-me.js:25
console.log("Goal: a page where my family shares recipes";
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

SyntaxError: missing ) after argument list
```

Nothing printed before the error, not even `ABOUT ME`. A `SyntaxError` means Node could not read the file as JavaScript, so it ran none of it, even the lines that were fine. Use the error, not your memory of what you deleted: the line number says line 25, the carets underline the quoted text, and the message says a `)` is missing after it. Put the `)` back, save, and run. Then write two lines in your notebook: the name and message of each error from steps 8 and 9, and the difference in what printed before each one.

© 2026 559 Solutions LLC

10. **Explain it.** With the script working, read the whole file to yourself, out loud, from top to bottom. For each comment, say why it is there. For each statement, say what it prints and why it prints at that point and not earlier. If there is a line you cannot explain, reread the Core Concept it comes from.

11. **Check the script.** Tick each box.

    - [ ] `about-me.js` is in `my-site`, next to `index.html`, not in `practice-05`
    - [ ] The prompt ends in `my-site>`, and `node about-me.js` prints three sections in the order from the brief
    - [ ] Every heading is in capital letters with a divider above and below; the top and bottom of the card are `=` signs
    - [ ] There is exactly one blank line before each of the two later sections, and nowhere else
    - [ ] Every fact line has a label, a colon and a space
    - [ ] The file starts with two comments, and each section starts with its own comment
    - [ ] Every statement ends with a semicolon
    - [ ] The output matches the plan you wrote in step 2, or you know why it changed
    - [ ] Nothing in the script is private

12. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and save the script with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your script works on your computer. Now you save it to GitHub with the same save routine as in Modules 1 to 4: `git status`, `git add .`, `git commit`, `git push`, then check the live site. What is new this time is the kind of file. `about-me.js` is not a page, and no page uses it. Git saves it all the same, and GitHub Pages publishes it as it is. Opening its address shows the code in the browser, and nothing runs it.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 4.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **See what changed.** Type:

   ```
   git status
   ```

   Under **Untracked files** you should see one line: `about-me.js`. Git has never saved it before. No page is listed, because no page changed. If you see a page or `styles.css` under **Changes not staged for commit**, you changed it by accident or left work from an earlier module unsaved: open it and check before you go on. Your `practice-05` folder never shows up here, because it is outside `my-site`.

3. **Stage the change.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see `new file: about-me.js`.

4. **Commit (NEW: a file that is not a page).** Type:

   ```
   git commit -m "Add an about-me script that Node runs"
   ```

   On the sample site, Git printed `1 file changed, 26 insertions(+)` on the second line and `create mode 100644 about-me.js` on the third. The 26 is the number of lines in the sample file: comments and blank lines count too. Your commit id and line count will differ. Git saves any file you give it, a page, a stylesheet, a picture or a script. It does not check whether a page uses it.

5. **Push.** Type:

   ```
   git push
   ```


© 2026 559 Solutions LLC

The last line ends in `main -> main`.

6. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 4 commit (or your latest one) under it.

7. **Check the live site (NEW: nothing changes).** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/` and press F5. Your Home page looks exactly as before, and so does every other page. That is correct. No page loads `about-me.js`, so adding it changed nothing a visitor sees.

8. **Open the script's own address (NEW: a published file that is not a page).** Open `https://your-username.github.io/my-site/about-me.js`. The browser shows your code as text, comments and all. Your browser may show it as text or offer to download it; either way nothing runs. If it shows the text, press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab: it is empty, because none of your `console.log()` lines ran. (This check was made on a local test server, not on GitHub Pages: Chromium showed the file's text and an empty Console.) Nobody runs this script except you, with Node. This is also why the brief said to keep the script free of private details: anyone with this address can read it.

9. **Run the saved copy.** Back in VS Code, type `node about-me.js` one more time. The script runs from the file on your computer. The copy on GitHub is your saved history and a backup. Node never reads it.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. Your live pages look the same as before, and the live address of `about-me.js` shows your code as text (or downloads it) without running it.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Each error below was made on purpose with the sample script and run with Node 22. The messages were captured on Linux, with the folder path changed to the course folder; on Windows, Node prints the same words with your own path. After each fix, save, and run `node about-me.js` again from the `my-site` folder.

1. "node" is not recognized

Symptom. You type `node about-me.js` (or `node --version`) and the terminal answers with a red error saying the term `node` is not recognized. Nothing from your script prints.

Why. The terminal cannot find the Node program. Either Node is not installed, or it was installed after VS Code was opened. A terminal inside VS Code gets its list of program locations from VS Code, and VS Code still has the list from before the install.

Fix. This is problem 1 of the Module 0 troubleshooting page. Close VS Code completely with **File → Exit**, open it again, choose **Terminal → New Terminal**, and type `node --version`. A new terminal in the same VS Code window is not enough. Still not recognized? Restart the computer. If it still fails, install Node again as in Module 0.

2. "Cannot find module" from the wrong folder

Symptom. Node prints an error that starts like this, and nothing from your script prints:

```
node:internal/modules/cjs/loader:1386
  throw err;
  ^

Error: Cannot find module 'C:\Users\you\Documents\code559\about-me.js'
```

Under it are some `at` lines and `code: 'MODULE_NOT_FOUND'`. The numbers in the first line depend on your Node version.

Why. `node about-me.js` looks for the file in the folder the terminal is in. Read the path in the message: it is the place Node looked. Here the terminal was in the course folder, `code559`, one level above `my-site`, so Node looked for `code559\about-me.js`, and there is no such file.

Fix. Check the prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal. Then run the command again. The error is the same if you type the file name wrong, for example `node aboutme.js`, so compare the name in the message with the name in the VS Code Explorer letter by letter.

© 2026 559 Solutions LLC

3. "Cannot find module" from the right folder

Symptom. The prompt ends in `my-site>`, but Node still prints `Error: Cannot find module 'C:\Users\you\Documents\code559\my-site\about-me.js'`. The file is right there: you can see your code.

Why. The file's real name is not `about-me.js`. The usual cause is a file created outside VS Code, for example in File Explorer with **New → Text Document**, while file extensions were hidden. You typed `about-me.js`, and Windows kept its own ending: the file is really `about-me.js.txt`. On the sample, a file named `about-me.js.txt` in `my-site` gave exactly this error. A file saved in another folder, such as `practice-05`, gives it too.

Fix. Look at the name in the VS Code Explorer, which shows the whole name. If it ends in `.txt`, click it, press F2, and rename it so it ends in `.js` only. Then turn on **View → Show → File name extensions** in File Explorer, as in Module 0, so this cannot hide again. If the file is in the wrong folder, drag it in the Explorer onto `my-site`, or create it again at the top level of `my-site`.

4. A SyntaxError, and nothing prints

Symptom. Not a single line of your card prints. Node shows a line of your code with a row of carets under it, then `SyntaxError:` and a message. With a missing closing quote on line 16 of the sample, it printed:

```
C:\Users\you\Documents\code559\my-site\about-me.js:16
console.log("Course: Code559 Front-End Developer Course);
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

SyntaxError: Invalid or unexpected token
```

With a missing `)` instead, the message was `SyntaxError: missing ) after argument list` (Project step 9).

Why. Before Node runs anything, it reads the whole file to check that it is valid JavaScript. One missing quote or parenthesis makes the file invalid, so Node runs none of it, not even the lines above the mistake. A missing closing quote is the most common cause: Node keeps reading the rest of the line as text, looking for the end of it.

Fix. Go to the line number in the first line of the error. Look where the carets start: on that line, every `"` needs a partner, and every `(` needs a `)` before the semicolon.

5. A ReferenceError partway through

Symptom. Some of your card prints, then it stops, and Node prints a line of your code with one caret under it and `ReferenceError: consol is not defined` (or another word in place of `consol`).

© 2026 559 Solutions LLC

Why. Every line above the mistake was fine, so Node ran those lines and printed them. Then it reached a name it does not know and stopped. In the sample, `consol.log` on line 23 printed 14 lines first (Project step 8). Capital letters count: `Console.log("b");` gave `ReferenceError: Console is not defined`, because JavaScript treats `Console` and `console` as two different names.

Fix. The last line that printed tells you where to look: the mistake is in the statement right after it. The first line of the error gives the same line number. Check the spelling of `console`, all lowercase, and fix it.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 6. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can explain that a program runs top to bottom, and predict the order of a script's output before I run it	☐	☐	☐	☐
I can say where JavaScript runs in this course, the browser's Console and Node, and choose the right one for a task	☐	☐	☐	☐
I can write statements that end in a semicolon, and comments that say what a part of the code is for	☐	☐	☐	☐
I can print text, numbers and blank lines with console.log(), and predict the difference between console.log(10 - 4) and console.log("10 - 4")	☐	☐	☐	☐
I can run a file with node from the right folder, and fix "Cannot find module" by reading the path in the message	☐	☐	☐	☐
I can open the DevTools Console with F12 or Ctrl+Shift+J, type an expression and read the answer	☐	☐	☐	☐
I can read a Node error from the top: the file and line, the code line, the caret, the name and message, and the file:line:column	☐	☐	☐	☐
I can tell a SyntaxError from a ReferenceError by what printed before it, and fix each one	☐	☐	☐	☐
I can plan a script with comments first and take it through Predict, Run, Modify, Create and Debug	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 8, 14 lines printed before the ReferenceError. In step 9, nothing printed before the SyntaxError. Explain the difference in your own words, and say what each error tells you about where to look.
2. Pick one prediction you got wrong in this module. What did you write down, what did the terminal or the Console show, and what did you not understand before that you understand now?

© 2026 559 Solutions LLC

3. A friend's `about-me.js` prints the `WHAT I AM LEARNING` section before `ABOUT ME`, and has no blank line between the last two sections. Without seeing the file, describe the two changes they need to make, and the statement each change is about.

© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **Why nothing printed in step 6.** A program runs from the top down, and Node stops at the first error. The typo was on line 2, the first statement in the file, so Node stopped before it had printed anything. The lines after line 2 were fine, but Node never reached them. Compare Project step 8: the same typo near the end of the file let 14 lines print first. (Core concepts 1 and 7; Exercise step 6; Project step 8)
2. **`console.log(10 - 4);` and `console.log("10 - 4");`.** Without quotes, `10 - 4` is a sum: JavaScript works it out first and prints `6`. With quotes, `"10 - 4"` is text, and `console.log()` prints its characters exactly as typed: `10 - 4`. (Core concept 4; Exercise step 5)
3. **Why `undefined` appears.** The Console does two things for each line you type. It runs the line, and then it shows the value the line hands back. `console.log("Hello from the Console")` prints its text while it runs. Then the Console shows what `console.log()` handed back, and it hands back no value, which JavaScript calls `undefined`. It is not an error: an error is shown in red and starts with `Uncaught`, as in Core concept 7. (Core concepts 6 and 7)

Common misconception

"An error message means I did something badly wrong. I should delete the line and start over." An error is a report, and it is free help. The ReferenceError in Exercise step 6 gave you, without any searching: the file, the line number (2), the line itself, a caret at the exact spot, the kind of problem (`ReferenceError`, a name JavaScript does not know) and the problem in words (`consol is not defined`). That is enough to fix a one-letter typo in seconds. Deleting the line throws all of that away, and if you retype it with the same mistake, you are back where you started. Read the message from the top, fix the one thing it points to, and run again. (Core concept 7; Exercise steps 6 and 7)

Exercise checks

- **Steps 2 and 3.** Three lines, in this order: `Hello, world!`, `This line runs second.` and `5`. The comment on line 1 prints nothing, and neither does the comment at the end of line 4. (Core concepts 3 to 5)
- **Step 4.** Four lines: the three from step 3, then `I added this line.`, last, because it is the last statement in the file. (Core concept 1)
- **Step 5.** The hours in a week, `console.log(7 * 24);`, prints `168`. `console.log(10 - 4);` prints `6`, and `console.log("10 - 4");` prints `10 - 4`. (Core concept 4)

© 2026 559 Solutions LLC

- **Step 6.** Nothing prints. Name: `ReferenceError`. Message: `consol is not defined`. The first line of the error and the first `at` line both point to line 2 of `hello.js`. (Core concept 7)
- **Step 7.** Nothing prints, not even line 2. Name: `SyntaxError`. Message: `Invalid or unexpected token`. The error points to line 3, with carets from the open quote to the end of the line. (Core concept 7)
- **Step 8.** `2 + 3` gives `5`. `"Hello, " + "world"` gives `"Hello, world"` (your Console may show single quotes). `consol.log("hi")` gives `Uncaught ReferenceError: consol is not defined`. (Core concepts 6 and 7)

Self-assessment questions

1. **Why 14 lines printed in step 8 but none in step 9.** A ReferenceError happens while the script runs. Node had already run every line above the typo, so those 14 lines printed, and it stopped only when it reached `consol` on line 23. A SyntaxError happens before the script runs: Node reads the whole file first, finds that it is not valid JavaScript, and runs none of it. Where to look: for a ReferenceError, the statement right after the last line that printed, which is also the line number in the error; for a SyntaxError, the line number and the carets, because the output gives no clue. (Core concept 7; Project steps 8 and 9)
2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before the run, quotes what the terminal or the Console really showed, and names the idea you understand now, for example that a SyntaxError stops the whole file, or that `""` prints an empty line.
3. **Fixing the friend's script.** First, move the whole `WHAT I AM LEARNING` section, from its comment to its last statement, below the `ABOUT ME` section, because statements run top to bottom. Second, add `console.log("");` as the first statement of the last section, before its first dashed divider, so the blank line prints between the two sections. (Core concept 1; Project steps 5 and 7)

Project checks

- **Step 3.** Nothing prints, and there is no error: the prompt comes straight back. A file with only comments has nothing to run.
- **Step 4.** Five lines: the `=` divider, `ABOUT ME`, the `=` divider, your name line and your fact line.
- **Step 5, continue alone.** The sample's Section 2. Seven statements, so the sample printed 12 lines in all.

```
// Section 2: what I am learning
console.log("");
console.log("------------------------------");
console.log("WHAT I AM LEARNING");
console.log("------------------------------");
```


© 2026 559 Solutions LLC

```
console.log("Course: Code559 Front-End Developer Course");
console.log("So far: HTML, CSS and JavaScript");
console.log("This week: running scripts with Node");
```

- **Step 6, create alone.** The sample's Section 3. The whole file now has 18 statements, so 18 lines print, as in the brief.

```
// Section 3: one thing I want to build
console.log("");
console.log("------------------------------");
console.log("ONE THING I WANT TO BUILD");
console.log("------------------------------");
console.log("Goal: a page where my family shares recipes");
console.log("==============================");
```

- **Step 7.** Without the first blank line, the dashed divider prints right under the last line of `ABOUT ME`. With the sections swapped, the building section and the closing `=` line print second, and the output ends with `This week: running scripts with Node`: the card has no bottom line. The statements ran in the order they sat in the file.
- **Step 8.** Sections 1 and 2 print, then the blank line and the first dashed divider of Section 3: 14 lines. Then `ReferenceError: consol is not defined`, pointing at line 23 in the sample.
- **Step 9.** None of the lines print. `SyntaxError: missing ) after argument list`, pointing at line 25 in the sample. A SyntaxError stops the whole file before it starts; a ReferenceError stops it only when Node reaches the bad line.
- **Step 11.** If you followed the brief, every box is ticked. The most common miss is a blank line in the wrong place: count the `console.log("");` statements. There should be exactly two, one at the start of Section 2 and one at the start of Section 3.

NEXT

Module 6: JavaScript Language Foundations.
Free lesson: 559solutions.com/code559/module-06-language-foundations/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC